

Lab Sheet 13: Release Management & CD

Objectives

By completing today's workshop, you will:

- Apply **Semantic Versioning (SemVer)** to real-world project scenarios.
 - Create annotated **Git Tags** and publish formal **GitHub Releases**.
 - Implement **Feature Flags** to decouple deployment from release in your backend.
 - Upgrade your CI pipeline to perform **Continuous Delivery (CD)** by pushing Docker images to a registry.
-

Introduction: Shipping the Product

For the past few weeks, you have focused on testing and refactoring your Ground Control Station codebase to ensure it is robust and clean. Today, we shift our focus from *building* the software to *releasing* it safely. You will learn how to bookmark specific versions of your code, hide unfinished features in production, and automate the delivery of your Docker containers.

Task 1: Semantic Versioning Scenarios

Goal: Practice determining the correct SemVer numbers.

The Concept: Semantic Versioning (**MAJOR.MINOR.PATCH**) communicates the exact nature of an update to other developers and users.

Action:

1. Create a file named **RELEASES.md** in your repository.
2. Assume the current live version of your Ground Control Station API is **v1.2.3**.
3. In your markdown file, write down what the **next version number** should be for each of the following independent scenarios, and briefly explain *why*:
 - **Scenario A:** You fix a bug where the robot crashed if the battery percentage dropped below zero. The API inputs/outputs stay exactly the same.
 - **Scenario B:** You add a brand new **/api/history** endpoint to retrieve past mission logs. All old endpoints work perfectly.
 - **Scenario C:** You completely redesign the JSON payload required for the **/api/move** endpoint. If old frontends try to send the old payload, they will receive a 422 Validation Error.

Task 2: Git Tagging & GitHub Releases

Goal: Create a formal, immutable snapshot of your codebase.

The Concept: A Git Tag bookmarks a specific commit. GitHub takes that tag and allows you to attach release notes, creating a formal "Release" that users can download.

Action:

1. Open your DevContainer terminal and ensure you are on the **main** branch and all your tests are passing.
2. Create an annotated Git tag for your first major release:

```
git tag -a v1.0.0 -m "Initial Release of the Ground Control Station"
```

3. Push the tag to GitHub (Note: `git push` does not push tags by default!):

```
git push origin v1.0.0
```

4. Go to your repository on GitHub.com. On the right-hand sidebar, click on **Releases**.
5. Click **Draft a new release**.
6. Select your **v1.0.0** tag from the dropdown. Click **Generate release notes** to automatically populate the description with your recent pull requests/commits.
7. Click **Publish release**. You now have a formal version of your software!

Task 3: Implementing Feature Flags

Goal: Deploy unfinished code to production without breaking the app for users.

The Concept: We use environment variables to toggle features on and off dynamically.

Action:

1. Open your backend `main.py` file.
2. Add Python's `os` module to read an environment variable. We convert the string to a boolean for easy `if` statement logic:

```
import os

# Read the feature flag, defaulting to "false" if not set
ENABLE_ADVANCED_STATS = os.getenv(
    "FF_ADVANCED_STATS", "false"
).lower() == "true"
```

3. Create a new endpoint wrapped in the Feature Flag:

```
@app.get("/api/experimental_stats")
def get_experimental_stats():
    if not ENABLE_ADVANCED_STATS:
        return {"error": "Feature not yet available."}, 404

    return {"status": "success", "data": "Top secret advanced stats!"}
```

4. **Test it:** Open your FastAPI docs (<http://localhost:8000/docs>). Try the endpoint. It should fail.
5. Open your `docker-compose.yml` file, find your backend service, and add the environment variable to turn it on:

```
environment:
  - FF_ADVANCED_STATS=true
```

6. Rebuild your containers (`docker compose up -d`). The feature is now active!

Task 4: Continuous Delivery to GHCR

Goal: Automate the creation of production-ready Docker artifacts.

The Concept: Currently, your GitHub Actions pipeline tests your code (CI). We will add a step to automatically build and push your Docker image to the **GitHub Container Registry (GHCR)** *only* when you push a new `v*.*.*` tag.

Action:

1. Open your existing GitHub Actions workflow file (`.github/workflows/ci.yml`).
2. At the very bottom of the file, add a new step to login and push to the registry. GitHub Actions automatically provides the `GITHUB_TOKEN` you need for authentication!

```
- name: Log in to the Container registry
  # Only run this step if the push is a semantic version tag!
  if: startsWith(github.ref, 'refs/tags/v')
  uses: docker/login-action@v2
  with:
    registry: ghcr.io
    username: ${ github.actor }
    password: ${ secrets.GITHUB_TOKEN }

- name: Build and push Docker image
  if: startsWith(github.ref, 'refs/tags/v')
  uses: docker/build-push-action@v4
  with:
    context: ./backend
    push: true
    tags: ghcr.io/${ github.repository }/backend:${ github.ref_name }
```

3. *Note: You may need to update the **permissions** block at the top of your YAML file to include **packages**: **write**.*
4. Commit and push this change to `main`.

5. Now, create and push a new tag: `git tag v1.1.0` then `git push origin v1.1.0`.
6. Watch your GitHub Actions pipeline. When it finishes, you will see a compiled Docker image appear in the "Packages" section of your GitHub repository!

Further Resources

- **Semantic Versioning Specs:** semver.org - *The official 11-point rulebook for SemVer.*
- **GitHub Packages Documentation:** docs.github.com/en/packages - *Learn how to manage and pull containers from GHCR.*